

GrowthPulse LoRaWAN System and Debugging Report

Last updated: June 11, 2026. Covers all work since the June 5 manual revision.

This document is the engineering record for the dual-transport (Wi-Fi and LoRaWAN) system: how it is built, how data flows in both directions, and a complete journal of every problem hit during bring-up with its root cause and fix. It is written so that a teammate, a future maintainer, or a grader can understand not just what the system does but why it is built the way it is, and what each fix actually solved.

It pairs with three other documents:

- **GrowthPulse Engineering Manual** for the original Wi-Fi-only architecture and the web app internals.
 - **GrowthPulse LoRaWAN Bring-Up Guide** for the one-time, by-hand gateway and The Things Stack setup.
 - **GrowthPulse LoRaWAN Cleanup and Reset** for the operational reset procedure.
-

Part 1. What we set out to build

GrowthPulse started as a Wi-Fi plant monitor. The goal over these three days was to make it a real consumer product that supports two ways of connecting, with the customer never touching any cloud console:

- **Wi-Fi nodes** for the home and greenhouse. A node joins the home network through a phone setup portal, registers itself with the cloud, and streams readings every few seconds.
- **LoRaWAN nodes** for fields and far gardens. A node joins through a long-range gateway and uplinks every few minutes. Range is measured in miles and a battery lasts months.

The hard requirements that shaped every decision:

1. **One firmware image for every board, with no per-board secrets.** A board figures out its own identity at first boot. This is what makes "mail a customer a board" possible.
 2. **The customer never logs into The Things Stack or Losant.** Adding a gateway, switching a node to LoRaWAN, and everything else happens in the GrowthPulse app.
 3. **The app always shows the truth.** Whatever link a node is actually using, and its real signal, is what the card shows, never a label the user picked.
-

Part 2. The two clouds and why there are two

GrowthPulse uses two managed clouds, each doing the thing it is best at.

Losant is the device cloud and the single source of truth the app reads from. Every node, whether Wi-Fi or LoRaWAN, ends up represented as one Losant device. The app only ever reads Losant. This is the key design choice that lets a single node switch transports and still be "one plant" in the app: both transports write to the same Losant device.

The Things Stack (TTS), on The Things Network community sandbox, is the LoRaWAN network server. It owns the radio side: gateways, OTAA join, and the LoRaWAN session. It does not store plant data. When a LoRaWAN uplink arrives, TTS forwards it to a webhook, which translates it into a Losant device-state update. So TTS is a bridge from the radio world into Losant, not a second source of truth.

A Wi-Fi node talks to Losant directly over MQTT. A LoRaWAN node talks to a gateway, which talks to TTS, which calls our webhook, which writes to Losant. Different paths in, same destination.

```
Wi-Fi node --MQTT--> Losant <--reads-- GrowthPulse app
LoRaWAN node --radio--> gateway --> TTS --webhook--> Netlify function --> Losant
```

Part 3. The Supabase layer (accounts and routing)

Supabase holds accounts and three small tables that glue the system together. None of these store sensor data; they store identity and routing.

- **device_registry**: maps a board's pairing code to its Losant device id. Written the first time a board self-provisions. This is what makes a factory-reset board reclaim the same Losant device instead of creating a duplicate: the pairing code is derived from the chip's hardware id and never changes, so the registry always returns the same device.
- **lorawan_devices** (v2 schema): maps a TTS end-device to the Losant device its uplinks should land on, and stores the board's OTAA AppKey so provisioning can reuse one TTS device per board. The board's Losant id is unique here, which enforces "one TTS device per board." See Part 8 for why that uniqueness matters.
- **gateways**: LoRaWAN gateways saved to the account so they persist across logins and browsers.

Row Level Security is on for these tables with no client policies, so the browser cannot read or write them. Only the serverless functions, using the Supabase service role, touch them. The service role bypasses RLS, which is exactly the intent.

Part 4. The Netlify functions (the backend)

Every function is a small serverless endpoint. The LoRaWAN-related ones, and what each does:

- **provision-device.js**: a board's first-boot self-registration. The board posts its pairing code plus a shared firmware token. The function looks the code up in `device_registry`; if new, it creates a Losant device (with the full attribute schema), registers the code, and returns a device id and a freshly minted access key scoped to just that device. As of v2.22 it also re-syncs the attribute schema on every call, so an older device gains any newly added attributes. This is the Wi-Fi self-provisioning path.
 - **provision-lorawan.js**: the LoRaWAN twin. Given a board's Losant id and a signed-in session, it either reuses the board's existing TTS device (looked up by Losant id, re-pushing its stored keys) or mints a new one through the TTS four-call API, stores the route, and pushes the OTAA keys to the board through a Losant command. As of v2.21 it is idempotent (one TTS device per board). As of v2.22 it clears the device's downlink queue and sets `resets_join_nonces` on reuse.
 - **lorawan-uplink.js**: the webhook TTS calls on every uplink. It decodes the 9-byte payload (using its own byte decoder so it does not depend on a per-device TTS formatter), resolves which Losant device the uplink belongs to from the `lorawan_devices` table, tags the reading with transport and signal, and writes it to Losant. As of v2.22 it logs the exact outcome of each delivery.
 - **lorawan-switch-wifi.js**: enqueues a one-byte 0x00 downlink to a node so it switches back to Wi-Fi. A LoRaWAN node is off the cloud's direct reach, so a downlink is the only way to reach it, and a downlink only lands during the node's next uplink window.
 - **register-gateway.js**: registers a customer's gateway on TTS under the GrowthPulse network so the customer never touches The Things Stack.
 - **device-state.js**: the read path the app polls. Returns the Losant composite state, including transport, wifiRssi, loraRssi, and loraSnr, so the app can show the true link.
-

Part 5. The combined firmware

One image, `firmware/GP_Combined/GP_Combined.ino`, version 4.2. A saved mode flag in flash (NVS) decides the boot path.

Wi-Fi mode (default): connect to saved Wi-Fi or open the phone setup portal; self-provision a Losant identity over HTTPS if none is saved; stream telemetry to Losant over MQTT every few seconds. Telemetry now stamps `transport: "wifi"`.

LoRaWAN mode: bring up the SX1262 radio with the Heltec V3's specific TCXO voltage and DIO2-as-RF-switch setting; load OTAA keys from NVS; join through any gateway in range; uplink a 9-byte payload every interval. The payload packs soil temperature, air temperature, humidity, raw soil, soil moisture percent, and battery percent. A one-byte 0x00 downlink switches the node back to Wi-Fi.

Mode switching happens three ways: the setup portal's Connection field, a Losant `setMode` command (while on Wi-Fi), and a LoRaWAN downlink (while on LoRaWAN). Setting the mode writes NVS and reboots into the new path.

No per-board secrets are in the image. Wi-Fi units self-provision; LoRaWAN keys are pushed at provisioning time and live only in NVS. The one shared value is the firmware provisioning token, kept as a placeholder in the repository copy and filled in only in a private build.

The PRG button is tiered: a single tap wakes from sleep; a double tap enters deep sleep (the soft power-off); a 3-second hold gracefully returns to Wi-Fi while keeping saved credentials; a 10-second hold is a full factory reset. The OLED shows which action will fire while the button is held.

The 9-byte LoRaWAN payload layout, for reference:

```
[0..1] soilTemperatureF * 10, int16 big-endian (0x8000 = sensor absent)
[2..3] airTemperatureF * 10, int16 big-endian
[4]    airHumidity percent,      uint8
[5..6] soilRaw 0..4095,         uint16 big-endian
[7]    soilMoisturePercent,      uint8
[8]    batteryPct 0..100,        uint8 (0xFF = unknown)
```

Part 6. The complete data path, both directions

Wi-Fi node, end to end. Board boots in Wi-Fi mode, connects to the home network, posts its pairing code to provision-device, receives a Losant device id and key, connects to Losant over MQTT, and publishes state every few seconds. The app polls device-state, reads the composite state, sees `transport: "wifi"` and a `wifiRssi`, and shows Wi-Fi with signal.

LoRaWAN node, end to end. Board boots in LoRaWAN mode, joins through a gateway via OTAA, and uplinks the 9-byte payload. The gateway forwards to TTS. TTS forwards the uplink to lorawan-uplink. The function decodes the bytes, looks up the route (TTS device id to Losant device id), tags the reading with `transport: "lorawan"` plus RSSI and SNR, and writes it to the same Losant device the board used over Wi-Fi. The app polls device-state, sees `transport: "lorawan"` and a `loraRssi`, and shows LoRaWAN with signal. Same Losant device, same plant card, different path in.

Part 7. Transport switching, both directions

Wi-Fi to LoRaWAN. The app calls provision-lorawan with the board's Losant id. The backend ensures a TTS device exists for that board and pushes the OTAA keys to the board through a Losant command. The board is on Wi-Fi at this moment, so it receives the command, saves the keys to NVS, and reboots into LoRaWAN, where it joins and uplinks. This direction works because the board is reachable on Losant while on Wi-Fi.

LoRaWAN to Wi-Fi. This is the hard direction. A LoRaWAN node is not on Losant's MQTT, so a Losant command cannot reach it. The only path down to the node is a LoRaWAN downlink, and a class-A node only listens during the brief window right after it uplinks. So lorawan-switch-wifi enqueues a one-byte 0x00 downlink, TTS holds it until the node's next uplink, delivers it in the receive window, and the firmware reboots into Wi-Fi. The latency of this switch is therefore bounded by the uplink interval, which is why the interval matters (Part 9, issue 12).

Part 8. Why "one TTS device per board" matters

This was the single most important architectural correction. The first version of provision-lorawan minted a brand-new TTS device, with a new DevEUI, on every Wi-Fi-to-LoRaWAN switch. Over a few test cycles a single board accumulated more than a dozen TTS devices, almost all of them dead. The routing table had several rows for one board, and the newest row pointed at a device the node had never actually joined.

The consequences cascaded:

- The switch-back-to-Wi-Fi downlink was aimed at the newest (dead) device, which had no session, so The Things Stack rejected it with `no_device_session`.
- The uplink webhook could route a real uplink to the wrong Losant device.

The fix (v2.21) made provisioning idempotent. The backend looks the board up by its Losant id, and if it already has a TTS identity it reuses it and re-pushes the same stored keys instead of creating a new device. The `lorawan_devices` table gained an `app_key` column (so keys can be re-pushed) and a uniqueness constraint on the Losant id (so a board can map to exactly one TTS device). The v2 migration clears the old orphan rows.

The lesson: a physical board must have exactly one stable network identity, and provisioning must be idempotent, or every downlink target and route becomes ambiguous.

Part 9. The debugging journal

Every problem hit during these three days, in roughly the order encountered, each with the symptom, the root cause, and the fix. This is the part to read to understand why the firmware and backend look the way they do.

Hardware and power

Issue 1. Battery always read 0% and the unit showed AC even on battery.

Root cause: this board's battery-sense control pin (GPIO37) is wired inverted from the documented Heltec reference, so the firmware was disabling the divider exactly when it meant to enable it. Fix (firmware v3.7): drive the control pin to the correct polarity and average 16 ADC samples for a stable reading.

Issue 2. No charging indicator.

Root cause: the Heltec V3 exposes no charge-status pin, so there is nothing to read directly. Fix (v3.7): infer charge state from the battery voltage trend and the charger's top-of-charge voltage, and report it. The OLED shows "Charging +" and the app shows a charging bolt.

Issue 3. Battery percent leaped from 44% to 65% the moment a charger was plugged in.

Root cause: a charger inflates the pack's measured terminal voltage instantly, which the naive percent curve read as a sudden jump. Fix (v3.8): smooth the percent so it creeps at most 1% per reading, rising only while charging and easing back toward the true level on battery.

Issue 4. No way to turn the unit off without unplugging.

Root cause: the board has no hardware power switch. Fix (v3.9): a double-tap of PRG drops the unit into deep sleep (microamp draw, screen off), and a single press or RESET wakes it. Waking from sleep no longer triggers the boot-time Wi-Fi wipe.

Issue 5. With no battery installed but on USB, the unit showed a phantom climbing percent.

Root cause: with no pack present, the charge chip holds the sense line near a full battery's voltage, and the board has no battery-detect pin to tell "no battery" from "full battery." This cannot be fully solved in software. Fix (v3.10): when the sense line sits pinned at the top and is no longer rising, report AC power instead of a misleading percent. The honest hardware fix is issue 6.

Issue 6. True USB-versus-battery detection (optional hardware mod).

Root cause: software alone cannot distinguish "no battery on USB" from "full battery." Fix (v3.11): an optional 100k/100k voltage divider from the board's 5V pin to ground, with the midpoint on GPIO7. On USB the midpoint reads about 2.5V; on battery it reads 0V. With `USE_VBUS_SENSE` set to 1, the firmware reads a real plugged-in signal. The wire is tapped from the board's labeled 5V pin, not from inside the USB-C cable. Default off, so boards without the wire are unchanged.

Web app honesty

Issue 7. The subscription plan reset to Free on every login.

Root cause: the tier was stored only in the browser. Fix (v2.16): persist the tier on the account (Supabase user metadata) so it follows the user across logins and devices.

Issue 8. LoRaWAN was faked.

Root cause: interim code forced every node to display Wi-Fi and carried "in development" notes. Fix (v2.17): the app now derives the true link from telemetry. A Wi-Fi build reports Wi-Fi; a LoRaWAN build that joins through a gateway shows as LoRaWAN, with real RSSI and SNR.

LoRaWAN radio and The Things Stack

Issue 9. The gateway was online but the node's joins failed with -1116 (no join accept), and the gateway "heard nothing."

Root cause: a sub-band mismatch. The ThinkNode G1 gateway shipped on US915 FSB1 (channels 0 to 7), while The Things Network uses FSB2 (channels 8 to 15). The gateway literally could not hear the node. Fix:

edit the gateway's channel plan (`global_conf_US915.json`) to FSB2 and make it permanent, since the generator only re-links the file.

Issue 10. The combined firmware joined but the first uplink failed with -1101 (session not active).

Root cause: in `RadioLib`, `begin0TAA` only sets the keys; `activate0TAA` performs the actual join. The combined image was calling only `begin0TAA`, so keys were set but the device never joined. Fix (v4.1): add the `activate0TAA` retry loop and use the 8-argument `sendReceive` form, matching the proven standalone LoRaWAN sketch.

Issue 11. provision-lorawan failed with "Identity Server create failed, 404 route_not_found" against the nam1 cluster.

Root cause: on The Things Network, the Identity Server (which registers devices and gateways) is centralized on the eu1 cluster, while a device's Network, Join, and Application servers run on the operating cluster (nam1 for North America). Fix: send the device-registration call to eu1, and the Join, Network, and Application Server calls to nam1.

Issue 12. "Forbidden path(s) in field mask" when configuring the device on TTS.

Root cause: each TTS server only accepts the field-mask paths it owns. Sending a Network Server a path that belongs to the Application Server is rejected. Fix: trim the Network Server field mask to the MAC and PHY paths, and send the Application Server an empty field mask.

The big LoRaWAN-to-app failures

Issue 13. After switching a node to LoRaWAN, the app card stayed on Wi-Fi.

This had several layered causes that were uncovered and fixed in sequence. The final, root cause is issue 16. Along the way:

- Auto-provisioned TTS devices had no per-device payload formatter, so the webhook received no decoded payload and bailed with 422. Fix (v2.20): the webhook decodes the 9 raw bytes itself, so it never depends on a TTS formatter.
- The route store could fail silently because the code only caught network errors, not a non-2xx Supabase response. Fix (v2.21): check the response status explicitly and surface the failure.

Issue 14. Switching back to Wi-Fi failed with `no_device_session`.

Root cause: orphan TTS devices. The downlink was aimed at a device the node had never joined, because provisioning minted a new DevEUI on every switch. Fix (v2.21): idempotent provisioning, one stable TTS device per board. See Part 8.

Issue 15. The Wi-Fi switch button never fired because it was gated on the live card state.

Root cause: the app only attempted the switch-to-Wi-Fi downlink if it already believed the link was LoRaWAN, but the card was stuck on Wi-Fi (issue 13), so the button was permanently inert. A chicken-and-egg deadlock. Fix: gate the switch on the saved transport, not the live reading, so it always fires and always returns a real result.

Issue 16. The app card never left Wi-Fi, and The Things Stack auto-deactivated the webhook.

This was the true root cause behind the card never showing LoRaWAN, and it took reading the webhook's

deactivation banner to find it. Losant rejects an entire state report if it contains an attribute the device does not define. The board's Losant device had been created before the `loraRssi`, `loraSnr`, and `transport` attributes were added to the provisioning schema, so it lacked them. Wi-Fi reports carry no LoRa fields and were accepted, but every LoRaWAN report carried `loraRssi` and `loraSnr` and was rejected with a 502. The repeated 502s caused The Things Stack to disable the webhook (its circuit breaker after several failed forwards), which is why the Netlify log looked empty. Fix (v2.22): provision-device now syncs the full attribute schema on every provision, not just on create, so an existing device gains the missing attributes on its next check-in. The webhook also now logs the exact 200/401/404/422/502 outcome so this class of failure is diagnosable in one glance.

The bounce loop and the stuck node

Issue 17. The node bounced endlessly between Wi-Fi and LoRaWAN, and sometimes wiped its own Wi-Fi.

Root cause: stale cloud commands. A `provisionLoRa` or `factoryReset` command would be delivered the instant the board reconnected to the cloud, before the user did anything, pulling it back into a mode change or wiping its credentials. Fix (v4.1): the firmware ignores `provisionLoRa`, `factoryReset`, and `setMode` for the first 12 seconds after connecting. A real command from the user arrives during steady operation; one delivered the instant a device connects is a stale replay.

Issue 18. A node stuck on LoRaWAN could not be recovered, and the 3-second button wiped Wi-Fi.

Root cause: to provision or switch a node, the backend pushes keys through Losant, which only works while the node is on Wi-Fi. A node stuck on LoRaWAN is unreachable that way, and the only escape hatch (the PRG button) used to perform a destructive Wi-Fi wipe. Fix (v4.1): a 3-second hold now gracefully returns to Wi-Fi while keeping saved credentials, and only a 10-second hold does the full reset. The deadlock is broken by a non-destructive button.

Issue 19. After a factory reset, the board appeared to get a different Losant identity, so the LoRaWAN route pointed at an orphaned device.

Root cause investigation showed the pairing code is derived from the chip's hardware id and is therefore stable, so a reset board does reclaim the same Losant device through provision-device. The apparent identity change was a symptom of the orphan-device problem (issue 14), not a real identity change. The takeaway, captured in the runbook: stop factory-resetting during testing, because each reset triggers a fresh provision cycle that, before idempotency, churned routing.

The last two rough edges

Issue 20. A stale 0x00 downlink bounced the node back to Wi-Fi right after every join.

Root cause: a leftover switch-to-Wi-Fi downlink, queued during earlier testing, sat in the TTS queue and was delivered on the first uplink after each join, kicking the node back to Wi-Fi before it could settle. Fix (v4.2): provision-lorawan clears the device's downlink queue when it provisions, so no stale downlink survives a switch to LoRaWAN. The firmware's earlier 20-second guard, which had also been suppressing legitimate switches, was removed in favor of this cleaner backend fix.

Issue 21. Joins took eight retries and about two minutes after a reflash.

Root cause: DevNonce. After a reflash wipes NVS, the board's join counter restarts at zero, but The Things Stack remembers the highest counter it has seen and silently drops every join until the board's counter climbs back past it. Each retry increments the counter by one, so a board that had reached eight needed eight retries. Fix (v4.1, v4.2): set `resets_join_nonces` on the device so The Things Stack accepts a reset counter and the node joins on the first try, and drop the retry interval from 30 seconds to 10 seconds so any residual catch-up is fast.

Issue 22. Switch-to-Wi-Fi from the app took up to 15 minutes.

Root cause: this is the class-A LoRaWAN reality, not a bug. A downlink can only reach the node during the short window right after it uplinks, and the production uplink interval was 15 minutes (chosen for The Things Network fair-use airspace). So a queued switch could wait up to a full interval. Fix (v4.2): lower the interval to 60 seconds for bench and demo use, with a documented note to raise it back toward 15 minutes for production fair-use compliance.

Operational notes that bit us

- **Netlify environment variables only take effect on a redeploy.** Setting a variable and expecting the running site to pick it up does not work; trigger a deploy.
- **A stale `.git/index.lock`** recurred on the development machine and blocked commits. The fix is `rm -f .git/index.lock` before committing.
- **The Things Stack webhook health is a circuit breaker.** A healthy-looking webhook that has been auto-deactivated for repeated failures will show empty function logs, which can read as "the webhook is not firing" when in fact it is firing and failing. Always check the webhook's status banner.

Part 10. How to recover a node to a known-good state

If a unit is misbehaving (stuck, bouncing, or showing the wrong link), this is the clean reset sequence. It avoids the factory-reset identity churn.

1. Get the node onto Wi-Fi and confirm it is online. If it is stuck on LoRaWAN, hold PRG for 3 seconds (graceful return to Wi-Fi, keeps credentials). If Wi-Fi was already wiped, reconnect through the setup hotspot; the board reclaims its same Losant device automatically.
2. Confirm the app card shows the board on Wi-Fi and online. This proves the board and the card are bound to the same Losant device.
3. Switch to LoRaWAN in the app once. With idempotent provisioning, this reuses the one TTS device, clears its downlink queue, sets `resets_join_nonces`, and pushes the keys. The node joins on the first try and holds.
4. To go back, switch to Wi-Fi. The 0x00 downlink lands on the node's next uplink (within the uplink interval) and it reboots to Wi-Fi.

The golden rule: a switch to LoRaWAN must start from a node that is on Wi-Fi and online, because that is the only time the cloud can hand it keys.

Part 11. Current status

- The full round trip works: a board joins LoRaWAN, its uplinks route through The Things Stack and the webhook to its Losant device, and the app card shows LoRaWAN with live RSSI and SNR. This was verified on hardware (a card reading "LoRaWAN, -44 dBm, SNR 12.5").
- Provisioning is idempotent, with one stable TTS device per board.
- The webhook decodes payloads itself and logs every outcome.
- The firmware ignores stale commands, has a tiered non-destructive button, joins fast, and stamps an authoritative transport tag.
- The remaining production hardening item is to raise the LoRaWAN uplink interval back toward 15 minutes before shipping, for fair-use compliance, accepting the corresponding switch-back latency.

GrowthPulse · Engineering Record · growthpulsecloud.com